



FurEver

Pet adoption & shelter management platform — full technical documentation.

Project type: Database systems final project — ASP.NET Core MVC web application

Stack: .NET 10 · EF Core 10 · SQL Server (Docker) · cookie auth

Document: Architecture, database design, feature set & design system

Generated: June 2026

Contents

1. Project Overview
2. Technology Stack
3. Application Architecture
4. Database Design — Tables & Schema
5. Database Logic — Triggers, Procedures & Constraints
6. Feature Set by Role
7. Routes & Controllers
8. Design System — Color, Type & UI

1. Project Overview

FurEver is a full-stack pet-adoption web application that doubles as a shelter management system. It lets the public browse adoptable animals and read each pet's medical history, lets registered adopters favorite pets and submit adoption applications, and gives shelter staff a complete administrative back office for managing pets, adoptions, adopters, veterinary records, vaccinations, and reporting.

The system is built around a relational SQL Server database whose integrity is enforced not only by the application but by database-level **triggers**, **stored procedures**, and **check constraints**. The core domain workflow — a pet moving from *Available* to *Reserved* to *Adopted* — is driven automatically by triggers on the Adoption table, so the data can never fall out of sync regardless of how a record is changed.

Design intent: the front end uses a deliberately warm, "handmade" visual identity — forest-green and crayon-box accents on a paper background, rounded display type, and soft sticker-style shadows — to make a shelter feel approachable rather than clinical.

2. Technology Stack

Runtime / Framework	.NET 10 — ASP.NET Core MVC (<code>Microsoft.NET.Sdk.Web</code>)
Language	C# with nullable reference types and implicit usings enabled
ORM	Entity Framework Core 10.0.9 (SQL Server provider)
Database	Microsoft SQL Server 2022, running in Docker (container <code>furever-sql</code>)
Authentication	ASP.NET Core cookie authentication (7-day sliding expiration)
Password hashing	BCrypt.Net-Next 4.2.0 (work factor 10)
View layer	Razor views (<code>.cshtml</code>) + tag helpers; MVC Areas for admin
Client libraries	Bootstrap, jQuery, jQuery-validation (unobtrusive)
Typography	Google Fonts — Nunito (body) + Shantell Sans (display)
File storage	Pet photos on disk in <code>wwwroot/uploads/</code>

NuGet packages

`BCrypt.Net-Next 4.2.0`

`Microsoft.EntityFrameworkCore.SqlServer 10.0.9`

Notable migration note

The schema was ported from an original MySQL design to SQL Server. Two MySQL features were re-implemented: the daily *EVENT* that flagged overdue vaccinations became the stored procedure `sp_update_overdue_vaccinations` (invoked at app startup and before vaccination queries, since SQL Server Express in Docker has no SQL Agent), and EF Core is explicitly told about the table triggers so it uses a save strategy that avoids the `OUTPUT` clause.

3. Application Architecture

Standard ASP.NET Core MVC layering with a clean separation between the public site and the admin back office (implemented as an MVC **Area**).

Request pipeline (Program.cs)

- HTTPS redirection + HSTS (production)
- Static files & mapped static assets
- Routing → Authentication → Authorization
- Two route maps:
`{area}/{controller}/{action}` and the default `{controller=Home}/{action=Index}`

Startup tasks

- Seeds a default admin (`admin@forever.com`) if none exists, hashing the password in C# with BCrypt
- Executes `sp_update_overdue_vaccinations` to refresh vaccination statuses
- Both wrapped in try/catch so the app still boots if the DB is not yet ready

Authentication & authorization model

Authentication is cookie-based. There are **two separate identity tables** — `Adopter` and `Admin` — and the login action checks both. Authorization is role-based via claims: routes are guarded with `[Authorize(Roles = "Adopter")]` or `[Authorize(Roles = "Admin")]`. Admins are redirected to the dashboard on login; the entire `Admin` area requires the Admin role.

Project layout

PATH	RESPONSIBILITY
<code>Models/</code>	EF Core entities (7) + view models (Account, Pet, Admin)
<code>Data/FurEverContext.cs</code>	DbContext: DbSet, unique indexes, trigger registration
<code>Controllers/</code>	Public + adopter controllers (Home, Account, Pets, Adoptions, Favorites, VetVisits)
<code>Areas/Admin/</code>	Admin back office (Dashboard, Pets, Adoptions, Adopters, VetVisits, Vaccinations, Reports)
<code>Views/</code>	Razor views + shared partials (<code>_Layout</code> , <code>_PetCard</code> , <code>_Glyph</code>)
<code>wwwroot/</code>	site.css design system, JS, client libs, uploaded pet photos
<code>database/</code>	<code>01_schema.sql</code> (DDL + triggers + procs), <code>02_seed.sql</code> (sample data)

Contact_No	nvarchar(15)	Required
Address	nvarchar(200)	Required
Housing_Type	nvarchar(20)	CHECK: House / Apartment / Condo / Farm
Has_Other_Pets	nvarchar(3)	CHECK: Yes / No
Has_Children	nvarchar(3)	CHECK: Yes / No
Experience_Level	nvarchar(20)	CHECK: First-time / Experienced

4.3 Adoption

Table `Adoption` — applications & completed adoptions. Carries four triggers. Indexed on Status, Pet_ID, Adopter_ID, dates.

COLUMN	TYPE	RULES
Adoption_ID	int identity	Primary key
Pet_ID	int	FK → Pet (ON DELETE CASCADE)
Adopter_ID	int	FK → Adopter (ON DELETE CASCADE)
Application_Date	date	Defaults to today
Adoption_Date	date	Nullable — auto-set by trigger on completion
Adoption_Fee	decimal(10,2)	Nullable — set by admin at approval
Contract_Signed	nvarchar(3)	CHECK: Yes / No — auto-Yes on completion
Status	nvarchar(20)	CHECK: Completed / Pending / Returned / Cancelled (default Pending)

4.4 Veterinary_Visit

Table `Veterinary_Visit` — medical record per pet. Indexed on Pet_ID.

COLUMN	TYPE	RULES
Visit_ID	int identity	Primary key
Pet_ID	int	FK → Pet (ON DELETE CASCADE)
Visit_Date	date	Required
Veterinarian_Name	nvarchar(100)	Required
Visit_Type	nvarchar(20)	CHECK: Checkup / Vaccination / Surgery / Treatment / Emergency
Weight	decimal(5,2)	Nullable (kg)
Temperature	decimal(4,2)	Nullable (°C)
Diagnosis	nvarchar(max)	Nullable
General_Notes	nvarchar(max)	Nullable
Procedure_Cost	decimal(10,2)	Required
Next_Visit_Date	date	Nullable

4.5 Vaccination

Table `Vaccination` — vaccine doses tied to a vet visit. Indexed on `Status`, `Next_Due_Date`.

COLUMN	TYPE	RULES
<code>Vaccination_ID</code>	int identity	Primary key
<code>Visit_ID</code>	int	FK → <code>Veterinary_Visit</code> (ON DELETE CASCADE)
<code>Vaccine_Name</code>	nvarchar(100)	Required
<code>Date_Administered</code>	date	Nullable
<code>Administered_By</code>	nvarchar(100)	Nullable
<code>Manufacturer</code>	nvarchar(50)	Nullable
<code>Next_Due_Date</code>	date	Nullable — drives Overdue logic
<code>Site</code>	nvarchar(50)	Nullable (injection site)
<code>Reaction</code>	nvarchar(max)	Nullable
<code>Status</code>	nvarchar(20)	CHECK: Completed / Scheduled / Overdue (default Scheduled)
<code>Cost</code>	decimal(10,2)	Nullable

4.6 Favorite & 4.7 Admin

Favorite

Join table (Adopter ↔ Pet). Unique on (`Adopter_ID`, `Pet_ID`).

<code>Favorite_ID</code>	int PK
<code>Adopter_ID</code>	FK, cascade
<code>Pet_ID</code>	FK, cascade
<code>Date_Added</code>	datetime2
<code>Notes</code>	nullable text

Admin

Staff accounts. Unique on Email. Seeded on startup.

<code>Admin_ID</code>	int PK
<code>Email</code>	unique, required
<code>Password_Hash</code>	BCrypt
<code>Full_Name</code>	required

5. Database Logic

Business rules live in the database itself so the data stays consistent no matter the entry path. This is the heart of the project's database-systems focus.

5.1 Triggers (on the Adoption & Pet tables)

TRIGGER	EVENT	WHAT IT DOES
<code>trg_adoption_validate_insert</code>	AFTER INSERT	Rejects (error 50001) an application unless the pet is <i>Available</i> ; if created directly as Completed, auto-fills Adoption_Date and Contract_Signed. Runs first.
<code>trg_adoption_after_insert</code>	AFTER INSERT	Moves the pet to <i>Reserved</i> (Pending) or <i>Adopted</i> (Completed) so it can't be double-booked.
<code>trg_adoption_after_update</code>	AFTER UPDATE	On transition to Completed, fills Adoption_Date / Contract_Signed; syncs Pet.Status: Completed→Adopted, Pending→Reserved, Returned/Cancelled→Available.
<code>trg_adoption_after_delete</code>	AFTER DELETE	If a reserved/adopted pet's adoption row is deleted, returns the pet to <i>Available</i> .
<code>trg_pet_cleanup_favorites</code>	AFTER UPDATE (Pet)	When a pet becomes <i>Adopted</i> , clears it from every adopter's favorites.

5.2 Stored procedures

PROCEDURE	PARAMETERS	PURPOSE
<code>sp_get_available_pets_by_species</code>	@p_species	Returns available pets of one species, newest arrivals first.
<code>sp_monthly_adoption_stats</code>	@p_year, @p_month	Aggregates a month: total/completed/pending/cancelled/returned counts + total revenue. Powers the admin report.
<code>sp_update_overdue_vaccinations</code>	—	Flips Scheduled vaccinations to Overdue when Next_Due_Date has passed. Runs at startup and before vaccination listings.

5.3 Check constraints (enumerations enforced in SQL)

TABLE	COLUMN	ALLOWED VALUES
Pet	Gender	Male, Female
Pet	Spayed_Neutered	Yes, No
Pet	Status	Available, Adopted, Reserved, Medical Hold
Adopter	Housing_Type	House, Apartment, Condo, Farm
Adopter	Has_Other_Pets / Has_Children	Yes, No
Adopter	Experience_Level	First-time, Experienced
Adoption	Status	Completed, Pending, Returned, Cancelled
Adoption	Contract_Signed	Yes, No
Veterinary_Visit	Visit_Type	Checkup, Vaccination, Surgery, Treatment, Emergency
Vaccination	Status	Completed, Scheduled, Overdue

The adoption lifecycle, end-to-end: an adopter applies → `validate_insert` confirms the pet is free → `after_insert` reserves the pet. An admin approves → `after_update` stamps the adoption date, marks the contract signed, sets the pet to Adopted, and `cleanup_favorites` removes it from everyone's wishlists. If the admin rejects, or the adoption is later returned, the pet flows back to Available automatically.

6. Feature Set by Role

Public visitor no login

- Home page with most-favorited pets and newest arrivals
- Browse all available pets; filter by species; search by name, breed, or temperament
- Pet detail page with photo, traits, special needs, and full medical history
- Public veterinary-visit transparency log (visits + vaccinations on record)
- Register an adopter account; log in

Registered adopter role: Adopter

- Favorite / un-favorite pets and attach private notes
- Submit an adoption application for an available pet (duplicates prevented)
- Track application status (Pending / Completed / Cancelled / Returned)
- Cancel one's own pending applications
- Edit profile: contact, address, housing, other pets, children, experience

Administrator role: Admin

Dashboard

KPI tiles (available / reserved / adopted / medical-hold pets, total adopters, pending applications, overdue vaccinations), recent applications, upcoming vet visits.

Pets

Full CRUD, status control, photo upload (JPEG/PNG/GIF/WebP, max 5 MB) with old-file cleanup.

Adoptions

Review all applications; Approve (set fee), Reject, or mark Returned — each driving the trigger-based status flow.

Adopters

Search adopters; view a profile with all their applications and favorited pets.

Vet visits & vaccinations

CRUD vet visits with vitals/cost/follow-ups; manage vaccinations with auto-overdue tracking.

Reports

Monthly adoption statistics & revenue (via stored procedure) plus species inventory breakdown.

7. Routes & Controllers

Public & adopter controllers

ROUTE	VERB	ACCESS	PURPOSE
/ Home/Index	GET	Public	Featured + new pets
/Pets/Index	GET	Public	Browse + filter + search
/Pets/Details/{id}	GET	Public	Pet profile + medical history
/VetVisits/Index	GET	Public	Transparency log
/Account/Register	GET POST	Public	Adopter sign-up
/Account/Login	GET POST	Public	Auth (Adopter or Admin)
/Account/Logout	POST	Auth	Sign out
/Account/Profile	GET POST	Adopter	View / edit profile
/Adoptions/Index	GET	Adopter	My applications
/Adoptions/Apply	POST	Adopter	Submit application
/Adoptions/Cancel	POST	Adopter	Cancel pending
/Favorites/Index	GET	Adopter	Saved pets
/Favorites/Add · Remove · UpdateNotes	POST	Adopter	Manage favorites

Admin area controllers all require Admin role

CONTROLLER	ACTIONS
Dashboard	Index (KPIs)
Pets	Index, Create, Edit, Delete
Adoptions	Index, Approve, Reject, Return
Adopters	Index, Details
VetVisits	Index, Details, Create, Edit, Delete
Vaccinations	Index, Create, Edit, Delete
Reports	Index (monthly stats by year/month)

8. Design System

A warm "handmade" identity: forest green with crayon-box accents on a soft paper background, rounded display type, hand-cut (slightly irregular) border radii, and soft sticker-style shadows. Every pet now has a real photograph.

8.1 Color palette

Defined as `oklch()` custom properties in `site.css`. Approximate hex shown for reference.



Status badges map directly to these tokens: **Available**→success green, **Pending/Reserved**→warning amber, **Adopted/Completed**→primary green, **Cancelled/Overdue/Rejected**→error red, **Medical Hold**→accent, neutral states→muted grey.

8.2 Typography

Display / headings	Shantell Sans — rounded, hand-drawn marker feel (weights 500/700/800)
Body / UI	Nunito — rounded humanist sans (weights 400/600/700/800)
Scale	Fluid <code>clamp()</code> steps from xs (~0.8rem) to 4xl (~3.4rem)
Headings accent	Hand-drawn amber "crayon underline" SVG beneath H1 and section titles

8.3 UI components & motifs

Buttons Pill shapes with an offset "drop" shadow that collapses on press. Variants: primary (green), accent (amber), secondary (outline), danger (red).	Pet cards 4:3 photo, status badge overlay, favorite count, name + meta line (breed · age · gender) and a one-line temperament note. Lift on hover.
Surfaces White cards with subtly irregular ("hand-cut") border radii, 2px soft borders, and offset sticker shadows on a paper-grain background.	Identity details Torn-paper edges on header/footer, hand-drawn SVG glyph set (dog, cat, rabbit, bird, heart...), forest-green sticky nav.

8.4 Accessibility & polish

- Status colors meet contrast targets against their backgrounds; white focus ring on dark green surfaces, accent ring elsewhere
- `prefers-reduced-motion` respected — transitions and smooth scroll disabled

- Full interactive-state coverage on inputs (focus rings) and buttons (hover / active / disabled)
- Semantic HTML with descriptive `alt` text on every pet photo; lazy-loaded images